Proiect ITBI 2025-2026

Echipa: The Pursuit of Happyness

Studenți: Gherghe Antonia și Popescu Luca

Problema aleasă (cerința)

MyLast (1–2 studenți) – Scrieți un program (script) shell care emulează comportamentul comenzilor last/lastb folosind în acest sens conținutul fișierului /var/log/auth.log[1-4] (Atenție: nu /var/log/wtmp sau /var/log/btmp care sunt fișiere binare). Implementarea trebuie să suporte minimal comportamentul comenzilor last/lastb indus de folosirea flag-urilor -n, -p, -s, -t. Puteți implementa două comenzi diferite, la fel ca în cazul last/lastb, sau puteți implementa o singură comandă care să emuleze comportamentul celor două.
Obs: Comanda less știe să decomprime fișiere de tip gzip, adică puteți rula comanda less /var/log/auth.log.3.gz și să obțineți rezultatul anticipat, dar alte comenzi, de pildă grep sau more, nu știu.

Descrierea proiectului

Proiectul MyLast (File-based Login Monitor) este un script utilitar dezvoltat în mediul shell, creat cu scopul de a oferi o metodă accesibilă de monitorizare și raportare a activității de autentificare pe sistemele Linux.

Spre deosebire de utilitarele clasice de sistem, precum last sau lastb, care accesează baze de date binare (wtmp, respectiv btmp), soluția propusă adoptă o abordare bazată pe fișiere text. Practic, programul „traduce” evenimentele brute înregistrate în jurnalul de securitate al sistemului, localizat în /var/log/auth.log, într-un format ușor de citit și interpretat.

Descrierea soluției

Obiective principale

1. **Monitorizarea accesului**: Scriptul permite vizualizarea cronologică atât a logărilor reușite (*Accepted*), cât și a celor eșuate (*Failed*), oferind astfel o imagine clară asupra securității sistemului.
2. **Procesarea arhivelor**: O caracteristică esențială este capacitatea de a analiza nu doar log-urile actuale, ci și pe cele vechi, arhivate automat de sistem sub format .gz. Acest lucru este realizat prin utilizarea comenzii zgrep, care permite citirea datelor comprimate fără a fi necesară dezarhivarea lor manuală în prealabil.
3. **Controlul volumului de date**: Utilizatorul are posibilitatea de a gestiona cantitatea de informație afișată prin intermediul flag-ului -n, funcție utilă în cazul sistemelor cu un volum mare de evenimente.

Logica de funcționare

În esență, programul scanează fișierele jurnal în căutarea unor „amprente” specifice:

1. Pentru a identifica **logările reușite**, scriptul filtrează mesajele de tipul „Accepted password” sau „session opened”.
2. Pentru **încercările eșuate**, acesta caută indicatori precum „Failed password” sau „authentication failure”.

Informațiile extrase sunt ulterior prelucrate prin awk pentru a extrage doar datele esențiale (utilizator, terminal, dată și oră) și sunt sortate invers cronologic, astfel încât cele mai recente activități să fie întotdeauna primele vizibile.

Implementare

Eficiența scriptului **MyLast** se bazează pe utilizarea conceptului pipeline. Această tehnică permite legarea mai multor comenzi simple, unde ieșirea unei comenzi devine intrarea pentru următoarea, transformând datele brute din log-uri într-un raport structurat.

Pentru a obține lista finală, datele trec prin următoarele etape de prelucrare:

1. Extracția cu *zgrep*: Spre deosebire de grep simplu, am utilizat zgrep pentru a putea procesa și fișierele comprimate .gz fără dezarhivare manuală, conform cerințelor specifice ale proiectului.
   Exemplu din cod:

```
date_brute=$(zgrep -E "$tipar" $fisiere_log 2>/dev/null
```

2. Sortarea cronologică (*sort*): Log-urile în /var/log/auth.log sunt scrise secvențial, cele mai vechi fiind la început. Pentru a emula comportamentul comenzii last, am folosit sort -r pentru o sortare inversă.
   Exemplu din cod:

```
date_brute=$(zgrep -E "$tipar" $fisiere_log 2>/dev/null | sort -r)
```

3. Limitarea rezultatelor (*head*): Când utilizatorul specifică flag-ul -n, folosim comanda head pentru a extrage doar numărul dorit de linii, economisind resurse de procesare.
   Exemplu din cod:

```
final=$(echo "$date_brute" | head -n "$limita")
```

4. Formatarea finală cu *awk*: Ultima etapă presupune curățarea liniei de log de informații redundante (precum numele host-ului) folosind awk.
   Exemplu din cod:

```
# folosim awk pentru a extrage coloanele exacte: user si data/ora
echo "$final" | awk '{print $9, "\t", $2, $1, $3}'
```

   În acest context, awk funcționează ca un motor de tokenizare care extrage coloana 9 (utilizatorul) și coloanele 1-3 (data și ora).

Gestionarea argumentelor (*case* și *shift*)

Implementarea suportă argumente multiple în linia de comandă prin utilizarea unei bucle while și a instrucțiunii case.

- Identificarea modului: Scriptul verifică dacă primul argument este last sau lastb pentru a seta tiparul de căutare (*Accepted* vs *Failed*).
- Utilizarea shift: Această comandă este esențială pentru a parcurge toți parametrii transmiși, permițând scriptului să „consume" argumentele procesate și să treacă la următoarele.

Importanța privilegiilor de administrator (*sudo*)

În ierarhia sistemului de fișiere, log-urile sunt considerate date sensibile, deoarece conțin detalii despre activitatea utilizatorilor și securitatea mașinii.

- Permisiuni restricționate: Fișierele din directorul /var/log/, cum este și auth.log, aparțin de regulă utilizatorului root și grupului adm.
- Accesul utilizatorului de rând: Un utilizator obișnuit va primi eroarea *„Permission denied"* dacă încearcă să citească un fișier pentru care nu are drepturi de citire.
- Soluția: Pentru a rula scriptul, este obligatorie utilizarea comenzii sudo, care permite executarea programului cu drepturi de administrator (*root*), oferind acces la fluxul de date din auth.log.

Locația și structura datelor (*/var/log/auth.log*)

- Scriptul accesează log-urile folosind calea absolută /var/log/auth.log pentru a asigura portabilitatea, indiferent de directorul curent în care se află utilizatorul.
- Dimensiunea și data ultimei modificări a log-urilor pot fi verificate folosind formatul lung al comenzii ls -l, aspect esențial pentru a confirma că sistemul chiar înregistrează activitate în acel moment.

Rotirea log-urilor și fișierele *.gz*

Sistemul folosește un mecanism de „rotire” a log-urilor pentru a preveni umplerea discului.

- Arhivarea: Când auth.log atinge o anumită dimensiune, conținutul vechi este mutat într-un fișier nou (ex: auth.log.1) sau arhivat (ex: auth.log.2.gz).
- Tratarea transparentă: Scriptul nostru folosește zgrep tocmai pentru a trata aceste arhive ca pe niște fișiere text obișnuite, permițând căutarea într-un istoric mai lung fără a ocupa spațiu suplimentar prin dezarhivare.

Pregătirea scriptului

- Crearea fișierului: Se folosește o comandă de tip editor (precum nano) pentru a salva codul într-un fișier numit mylast.sh.
- Drepturi de execuție: Implicit, un fișier nou creat nu poate fi rulat. Trebuie să îi acordăm permisiuni de execuție folosind comanda chmod.
- Localizarea: Pentru a rula un script din directorul curent fără ca acesta să fie în variabila $PATH, prefixăm numele cu ./.

Exemple de utilizare

- Comandă: sudo ./mylast.sh last

  Rezultat: Afișează toate sesiunile "Accepted" găsite în log-urile curente și arhivate.

- Comandă: sudo ./mylast.sh -n 5 last

  Rezultat: Afișează cele mai recente 5 intrări “Accepted”

- Comandă: sudo ./mylast.sh lastb

  Rezultat: Filtrează log-urile după tiparul "Failed password" sau "authentication failure".

Rezultate experimentale

```
┌──(anto㉿antonia)-[~]
└─$ sudo ./mylast.sh last
[sudo] password for anto:
utilizator | terminal | data si ora
──────────────────────────────────
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:43:14.536892-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:39:01.728032-05:00 CRON[103944]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:35:01.708015-05:00 CRON[102011]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:25:01.689321-05:00 CRON[97150]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:17:01.672829-05:00 CRON[93260]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:15:01.641203-05:00 CRON[92301]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:09:01.626125-05:00 CRON[89367]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:05:01.611588-05:00 CRON[87447]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:55:01.591486-05:00 CRON[82637]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:45:01.568640-05:00 CRON[77830]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:39:01.555629-05:00 CRON[74896]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:35:01.540170-05:00 CRON[72974]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:25:01.519157-05:00 CRON[68167]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:17:01.499971-05:00 CRON[64320]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:15:01.487347-05:00 CRON[63354]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:09:01.472496-05:00 CRON[60418]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T16:05:01.457981-05:00 CRON[58498]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T15:55:01.432740-05:00 CRON[53693]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T15:45:01.411166-05:00 CRON[48887]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T15:39:06.393898-05:00 CRON[46043]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T14:17:01.528456-05:00 CRON[42156]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T14:15:01.514820-05:00 CRON[41117]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T14:09:01.498564-05:00 CRON[38084]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T14:05:01.487110-05:00 CRON[36148]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T14:04:26.179735-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T13:55:01.468714-05:00 CRON[31235]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T13:51:19.389833-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T13:50:34.615428-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T13:50:03.492862-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T13:46:50.764084-05:00 sudo:
```

```
┌──(anto㉿antonia)-[~]
└─$ sudo ./mylast.sh -n 5 last
[sudo] password for anto:
utilizator | terminal | data si ora
──────────────────────────────────
root(uid=0)      antonia /var/log/auth.log:2026-01-10T18:05:35.282687-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T18:05:01.786660-05:00 CRON[116638]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:55:01.768445-05:00 CRON[111828]:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:47:59.390165-05:00 sudo:
root(uid=0)      antonia /var/log/auth.log:2026-01-10T17:45:01.743379-05:00 CRON[106967]:
```

```
┌──(anto㉿antonia)-[~]
└─$ sudo ./mylast.sh lastb
utilizator | terminal | data si ora
──────────────────────────────────
euid=1000        antonia /var/log/auth.log:2026-01-10T14:04:08.273626-05:00 xfce4-screensaver-dialog:

┌──(anto㉿antonia)-[~]
└─$
```

Concluzii

Proiectul MyLast reprezintă o aplicație practică a conceptelor fundamentale de administrare a sistemelor Linux. Așadar, utilizarea „conductelor de date” permite transformarea unor jurnale de sistem complexe și voluminoase în rapoarte esențiale, ușor de parcurs.